

# Curso de Go

*Aula 4 - Controle de fluxo em Go*

*Professor : Antônio Marcelo*

NÚCLEA  
ACADEMY

 **código\_** [3]  
**BRAZUCA**





# Nessa Aula nos Vamos Ver

- **Controle de fluxo em Go**
- **Entender como controlar a execução do programa com if, switch, for**
- **Boas práticas**



# O que é controle de fluxo

- **Controle de fluxo define como o programa toma decisões e repete ações**
- **Principais estruturas em Go:**
  - **if → decisões**
  - **switch → múltiplos caminhos**
  - **for → repetição**



# Estrutura if

- Usado para executar código baseado em condição

Exemplo de código

```
package main
import "fmt"

func main() {
    idade := 18

    if idade >= 18 {
        fmt.Println("Maior de idade")
    }
}
```

**Sem uso de parênteses na condição**



# Estrutura if com else e else if

- **Permite múltiplas decisões**

Exemplo de código

```
nota := 7
```

```
if nota >= 9 {  
    fmt.Println("Excelente")  
} else if nota >= 6 {  
    fmt.Println("Aprovado")  
} else {  
    fmt.Println("Reprovado")  
}
```





# if com variável inline

- Go permite declarar variável dentro do if
- A variável só existe dentro do bloco

Exemplo de código

```
if idade := 20; idade >= 18 {  
    fmt.Println("Maior de idade")  
}
```



# Estrutura switch

- Alternativa mais limpa para vários if
- Não precisa de break

Exemplo de código

```
dia := 3
```

```
switch dia {  
case 1:  
    fmt.Println("Domingo")  
case 2:  
    fmt.Println("Segunda")  
case 3:  
    fmt.Println("Terça")  
default:  
    fmt.Println("Outro dia")  
}
```



# switch sem variável

- Pode ser usado como if mais organizado

Exemplo de código

```
idade := 16
```

```
switch {  
case idade >= 18:  
    fmt.Println("Adulto")  
case idade >= 12:  
    fmt.Println("Adolescente")  
default:  
    fmt.Println("Criança")  
}
```





# Estrutura for

- **Única estrutura de repetição em Go**

Exemplo de código : If clássico

```
for i := 0; i < 5; i++ {  
    fmt.Println(i)  
}
```

Exemplo de código : Tipo While

```
i := 0  
for i < 5 {  
    fmt.Println(i)  
    i++  
}
```

Exemplo de código : Loop infinito:

```
for {  
    fmt.Println("Loop infinito")  
}
```



# Goto

- O goto em Go permite saltar diretamente para um rótulo dentro da mesma função, alterando o fluxo de execução de forma não sequencial
- Seu uso é permitido, mas restrito ao escopo da função, o que evita alguns abusos comuns de outras linguagens
- É considerado não idiomático na maioria dos casos, pois prejudica a legibilidade e manutenção do código
- Pode ser útil em situações específicas, como saída antecipada de múltiplos níveis de loop ou tratamento de erros complexos, mas deve ser usado com muita cautela



# Goto

- **Uso**

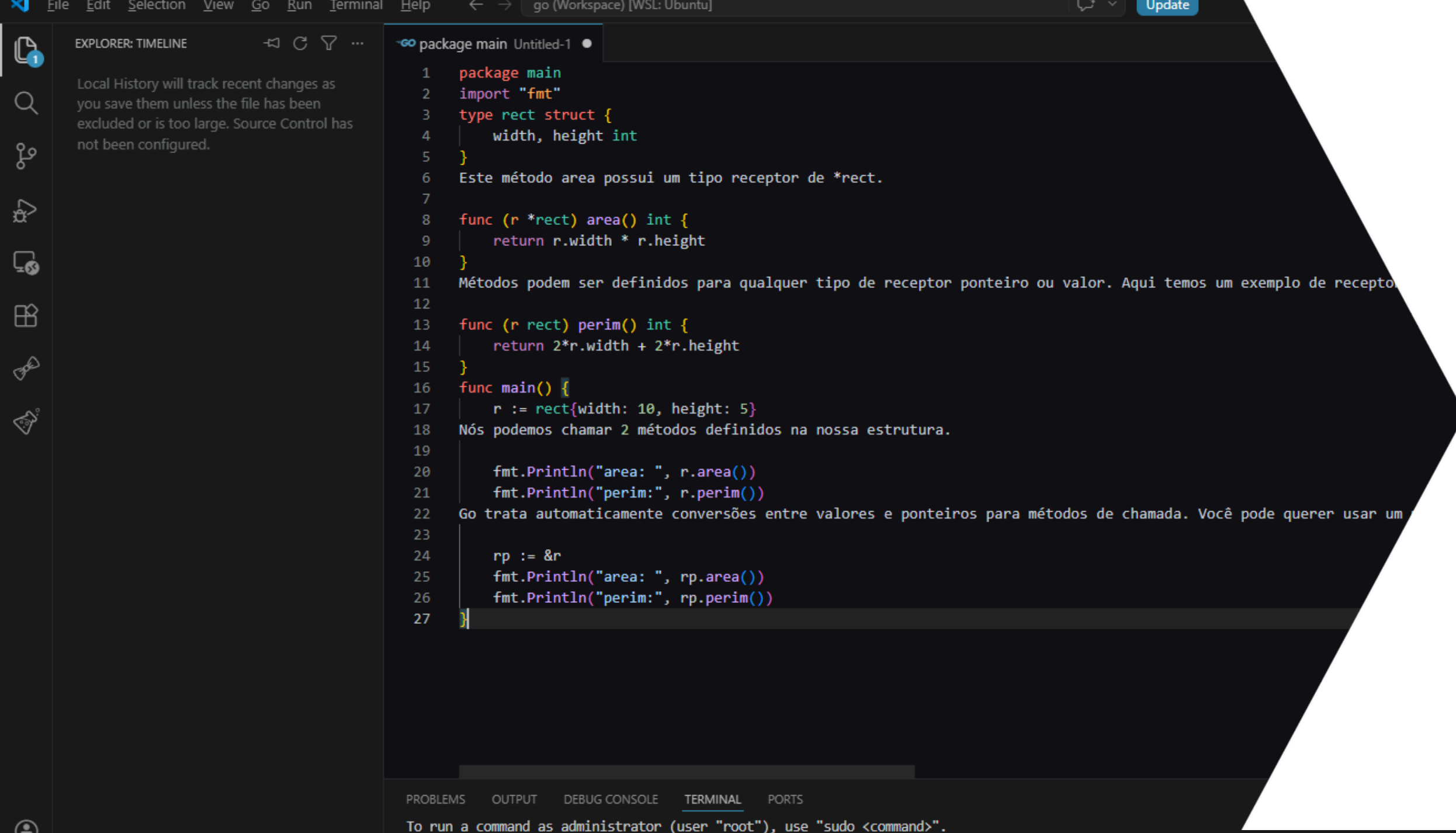
Exemplo de código : If clássico

```
func main() {  
    i := 0  
  
inicio:  
    if i < 3 {  
        fmt.Println(i)  
        i++  
        goto inicio  
    }  
}
```



# Boas Práticas

- Prefira if, switch e for no lugar de goto, mantendo o fluxo previsível e fácil de seguir
- Evite código confuso reduzindo aninhamentos excessivos e dividindo lógica complexa em funções menores
- Use nomes claros e descritivos para variáveis e funções, refletindo exatamente o propósito delas
- Mantenha blocos curtos e coesos, com uma única responsabilidade, facilitando leitura e manutenção
- Padronize o estilo do código (indentação, organização e nomenclatura) para consistência no projeto
- Comente apenas quando necessário, explicando o “porquê” e não o óbvio
- Evite duplicação de código reutilizando funções sempre que possível
- Escreva código pensando em quem vai ler depois, não apenas em quem está escrevendo agora



The screenshot shows a Go IDE with a dark theme. The Explorer sidebar on the left shows a file named 'package main' and a message: 'Local History will track recent changes as you save them unless the file has been excluded or is too large. Source Control has not been configured.' The main editor displays a Go program with the following code:

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de recepto
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

The bottom status bar shows tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active, displaying the message: 'To run a command as administrator (user "root"), use "sudo <command>".'

NÚCLEA  
ACADEMY

# Parte Prática

## Praticar com condições